COM3021 – Data Science at Scale

Software-hardware co-design

DR YANDA MENG

Introduction

- Specialised applications in areas such as AI/DS are pushing the boundaries of current hardware architectures;
- Impact on performance

  Specialised software on general hardware;

  General software on specialised hardware;
- Designing a new, specialised architecture is extremely expensive and therefore requires a large demand;
- Not a new field but more recently being pushed by AI and ML

Software-hardware co-design

- To deliver high performance computational demands (as Moore’s Law scaling slows down) we need both software and hardware to be specialised to each other
  - *A hardware designed specifically taking into account the characteristics of a software (algorithm, ML model)*
  - *A software (library, model) fine-tuned to this (yet to exist) hardware, taking full advantage of its capabilities*

Impact of AI and ML

- AI and ML workloads have some particularities

  Models based on tensor operations

  - *Certain models (e.g., CNN in image processing) will focus on dense tensors*
  - *Graph-based applications (e.g, recommendation systems) will work more with sparse tensors*

- Specialised hardware is necessary to meet these requirements

Impact of AI and ML

- AI/ML applications are becoming increasingly present in our daily lives
- AI/ML frameworks often run on the cloud
- As these models become more personalised, there is a pressure for the computation to be performed on devices
- Need for specialised software and hardware to deliver that

Edge computing

Beyond AI/ML

- Same trend is happening in other domains
  - *New classes of processing units used in datacentres*
    - *Infrastructure and data processing units (IPUs, DPUs)*
      - *Housekeeping, communication*
  - *Self-driving vehicles*

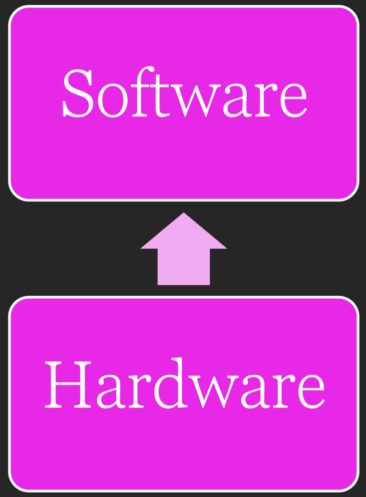
Bottom-up design

- Hardware/software

  Hardware is designed first focusing on a general concept of how it will be used

  AKA platform-based design

  A specialised software is designed for that piece of hardware

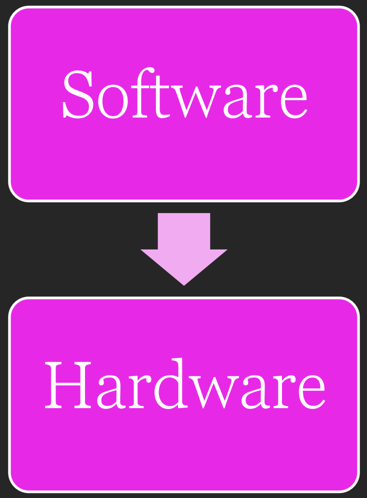


Top-down design

- Software/Hardware

  Software workloads drive the design of new hardware architectures

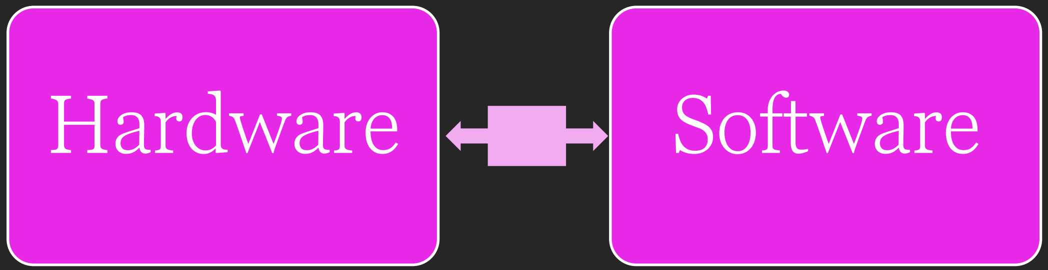
  Specialised AI architectures (e.g., Nvidia Volta - tensor cores specialised for tensor operations)



Hardware-software co-design

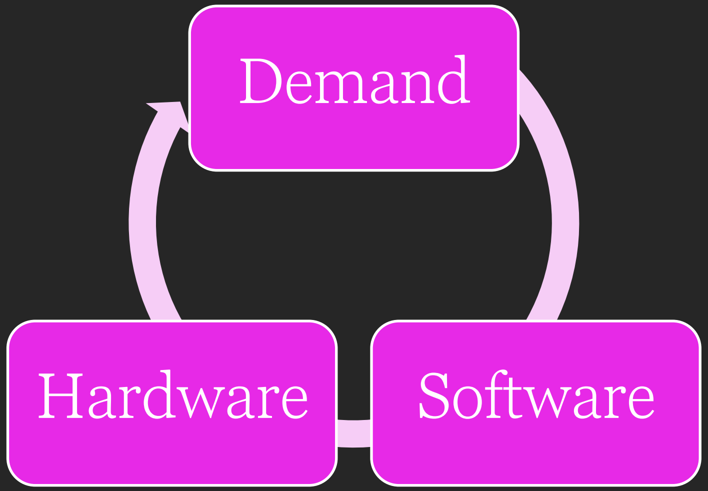
- Design the hardware and the software as a coupled process

  Designed, analysed and optimised together



Iterative process

- Neither approach is better than the other and sometimes they both walk together
- Demand creates the necessity for more specialised software which will drive the development of new architectures and more specialised software



Partitioning

- A crucial step in co-design is partitioning

  functions allocated between hardware and software

    Hardware (e.g., for speed and parallelism)

    Software (for flexibility and ease of updates)

Prototyping and Simulation

- Co-design often uses prototyping and simulation tools to model how the hardware and software will interact.
- This can involve the use of hardware description languages (HDLs) and software development tools to create accurate models of the system.

High-level synthesis

- Automated design process based on a high-level specification of the behaviour of that system

  Decoupled from the hardware implementation details

- A high-level specification the system is translated into a hardware specification design

  Hardware description language (HDL)

Platform-Based Design

- This approach involves using a predefined platform (a set of hardware and software components) as a starting point, which can significantly reduce design time and complexity.

Advantages

- Design-space Exploration

  Co-design allows designers to explore a wider range of design options and trade-offs between hardware and software, such as power consumption, performance, cost, and development time.

- Optimization

  Hardware/software co-design enables optimization at various levels, including system-level, architectural level, and algorithmic level, leading to more efficient designs.

Applications

- **Embedded Systems**: Co-design is particularly relevant in embedded systems, where hardware constraints (like size and power consumption) and the need for specialized functionality are common.
- **Consumer Electronics**: Devices like smartphones and smart appliances benefit from co-design, as it allows for the efficient integration of diverse functionalities.
- **Automotive Industry**: The increasing complexity of automotive electronics, including autonomous driving systems, makes co-design an essential approach.
- **Telecommunications**: In areas like 5G technology, co-design helps in implementing complex algorithms efficiently on hardware.